

Typesetting with *typst*

Roopesh O R

2026-09-12

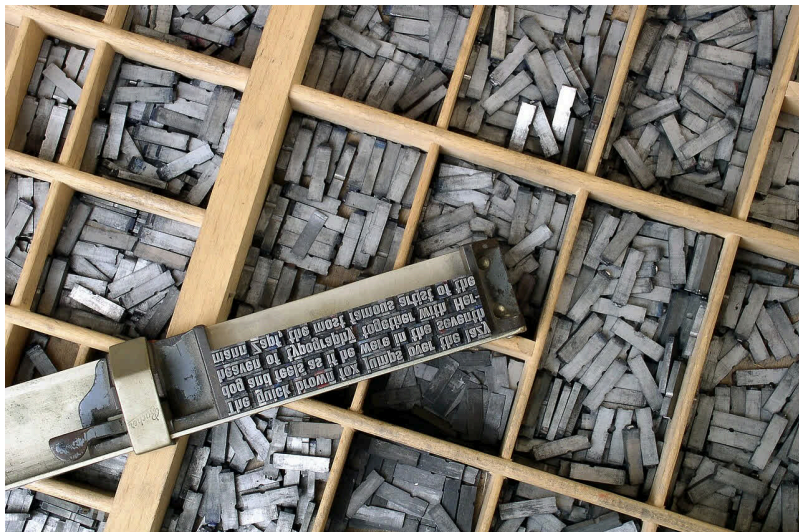
Outline

1. Introduction	3		
1.1 What is Typesetting?	4		
1.2 Various systems for Typesetting	5		
1.3 Why focus on structure	6		
1.4 Why Typst?	7		
2. Basic Componets	8		
2.1 Formatting	9		
2.2 Lists	10		
2.3 Alignment	11		
2.4 Heading	12		
2.5 Table	13		
2.6 Figures	14		
2.7 Column layout	15		
2.8 Grid layout	16		
2.9 Equations	17		
2.9.1 Aligning equations	20		
2.9.2 Matrix	21		
3. Styling and Settings	22		
3.1 Set & Show rule	23		
3.1.1 Test Code	23		
3.2 Page settings	24		
4. Scripting	26		
4.1 Variables & Conditional statemnts	28		
4.2 Loops	29		
4.3 Functions	31		
5. Packages	34		

1. Introduction

1.1 What is Typesetting?

Typesetting is the composition of text for publication, display, or distribution by means of arranging physical type (or sort) in mechanical systems or glyphs in digital systems representing characters

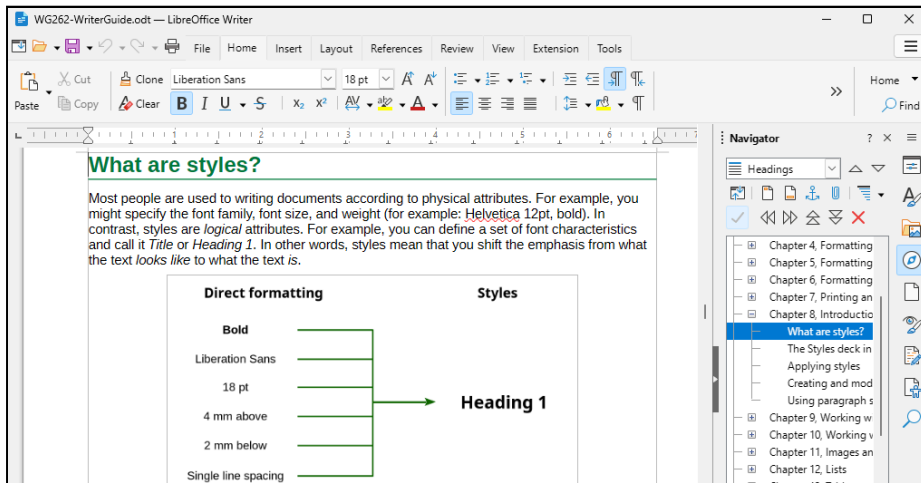


1.2 Various systems for Typesetting

WYSIWYG

(What you see is what you get)

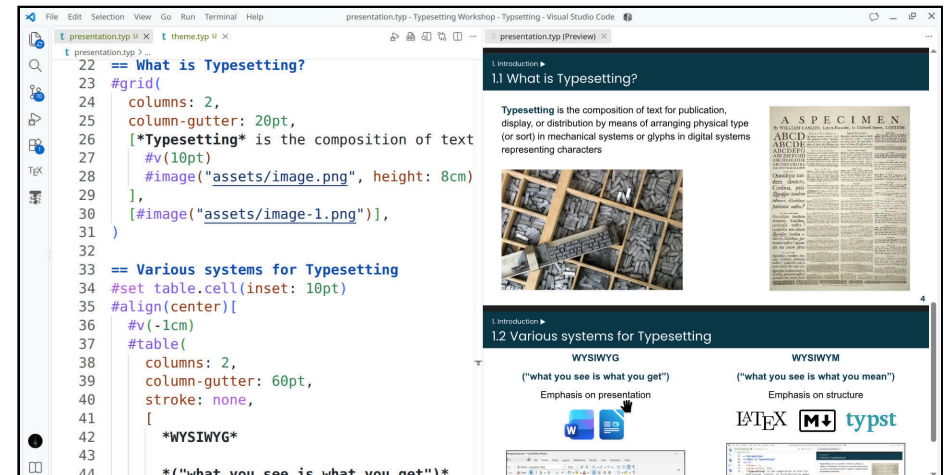
Emphasis on presentation



WYSIWYM

(what you see is what you mean)

Emphasis on structure/“semantics”



1.3 Why focus on structure

- Separation of content and presentation
- Change appearance at “scale”
- <https://typst.app/#how-does-it-work>

1.4 Why Typst?

- Extremely fast
- Can be exported to PDF, HTML, PNG
- Emerging replacement for traditional Typesetting systems
- Create figures, charts in Typst itself

2. Basic Componets

2.1 Formatting

Typst code	Output
Bold	Bold
<i>_Italic_</i>	<i>Italic</i>
<u>#underline[Italic]</u>	<u>Italic</u>
Inline code (monospaced): <code>`function()`</code>	Inline code (monospaced): <code>function()</code>
Block code: <pre>```py print("Hello " + str(10)) ```</pre>	Block code: <pre>print("Hello " + str(10))</pre>
<pre>#text(font: "Impact", size: 20pt, fill: red,)</pre> [Custom font]	Custom font

2.2 Lists

Unordered list:

- item 1
- item 2
- item 3

Unordered list:

- item 1
- item 2
- item 3

Ordered list:

- + item 1
- + item 2
- + item 3

Ordered list:

1. item 1
2. item 2
3. item 3

Nested list:

- + item 1
 - item 1.1
 - item 1.3
- + item 3

Nested list:

1. item 1
 - item 1.1
 - item 1.3
2. item 3

2.3 Alignment

- Text: justified or not
- Content: center, left, right

2.4 Heading

Supports unlimited nested headings, and automatic numbering also

```
#tile("Title")  
= Heading 1  
== Heading 2  
=== Heading 3  
==== Heading 4
```

Or:

```
#heading(level: 1)[Heading 1]  
#heading(level: 2)[Heading 2]  
#heading(level: 3)[Heading 3]  
#heading(level: 4)[Heading 4]
```

Title

Heading 1

Heading 2

Heading 3

Heading 4

2.5 Table

```
#table(
  columns: 3,
  [col 1], [col 2], [col 3],
  [1], [2], [3],
  [4], [1], [2],
)
```

col 1	col 2	col 3
1	2	3
4	1	2

```
#table(
  columns: (1fr, 3cm, 5cm),
  [*header*], [*Header2*], [*Header3*],
  [item1], [item2], [item3],
  [item1], [item2], [item3],
  [item1], [item2], [item3],
)
```

header	Header2	Header3
item1	item2	item3
item1	item2	item3
item1	item2	item3

2.6 Figures

```
#figure(  
  image("image.png", width: 13cm),  
  caption: [image caption],  
)
```

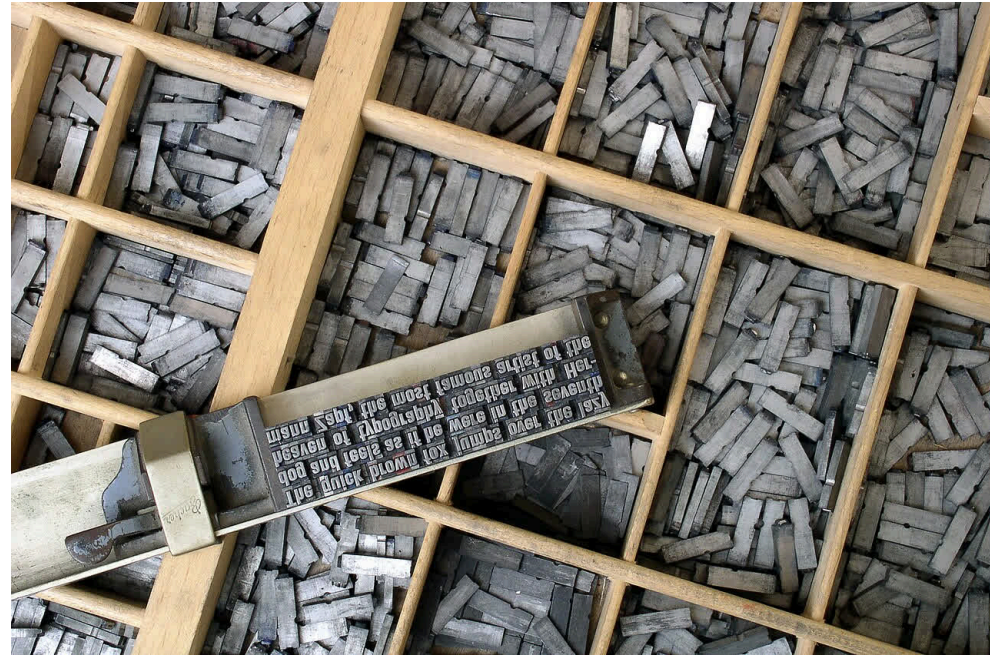


Figure 1: image caption

2.7 Column layout

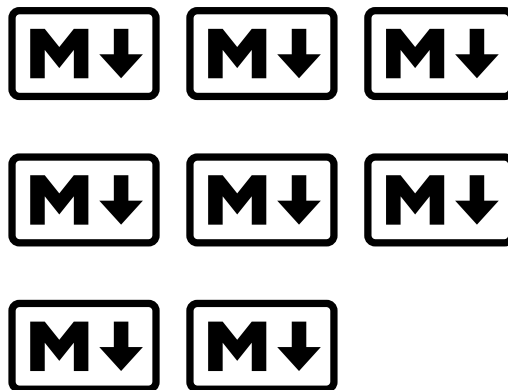
```
#columns(2) [  
  #lorem(100)  
]
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam

facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

2.8 Grid layout

```
#grid(  
  columns: 3,  
  row-gutter: 20pt,  
  column-gutter: 10pt,  
  [ #image("logo.svg") ],  
  [ #image("logo.svg") ],  
  [ #image("logo.svg") ],  
  [ #image("logo.svg") ],  
  [ #image("logo.svg") ],  
  [ #image("logo.svg") ],  
  [ #image("logo.svg") ],  
  [ #image("logo.svg") ],  
)
```



2.9 Equations

- inline mode: $\$x+y\$$: $x + y$
- block mode: $\$ x+y \$$:

$$x + y$$

2.9 Equations

\$ x+y \$	$x + y$
\$ \sin(x) \$	$\sin(x)$
\$ 1/x \$	$\frac{1}{x}$
\$ (x^2+y^2)/(2x) \$	$\frac{x^2 + y^2}{2x}$
\$ y = (-b \text{ plus.minus sqrt}(b^2 - 4a c))/(2a) \$	$y = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$
\$ \text{sum}_{(n=1)}^{\text{infinity}} 1/n^2 = \text{pi}^2/6 \$	$\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$

2.9 Equations

<pre>\$ lim_(A -> infinity) integral_0^A 1/(log_10(x))d x \$</pre>	$\lim_{A \rightarrow \infty} \int_0^A \frac{1}{\log_{10}(x)} dx$
<pre>\$ d^2/(d x^2) cos^2(x) \$</pre>	$\frac{d^2}{dx^2} \cos^2(x)$

2.9 Equations

2.9.1 Aligning equations

$y = (x+a)(x+b)$ $= x^2 + bx + ax + ab$ $= x^2 + ab + x(a+b)$	$y = (x+a)(x+b)$ $= x^2 + bx + ax + ab$ $= x^2 + ab + x(a+b)$
---	---

2.9 Equations

2.9.2 Matrix

```
$  
A = mat(  
  delim: "[",  
  1, 2, 4;  
  1, 3, 4;  
  5, 6, 7  
) mat(x; y; z, delim: "[")  
= mat(1; 2; 3)  
$
```

$$A = \begin{bmatrix} 1 & 2 & 4 \\ 1 & 3 & 4 \\ 5 & 6 & 7 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

3. Styling and Settings

3.1 Set & Show rule

- For customizing the appearance of Elements

```
#set text(  
  font: "New Computer Modern",  
  size: 16pt  
)  
#show heading.where(level: 3) : set text(  
  size: 20pt  
)  
#set list(marker: [--])  
=== Test Code  
#text[With set rules, you can style your  
document.]  
- list  
- list
```

3.1.1 Test Code

With set rules, you can style your document.

- list
- list

3.2 Page settings

```
#set page(  
  numbering: "a",           // numbering style  
  fill: rgb("#fff2f2")      // background color  
  background: image("logo.svg") // other background stuff like watermarks, etc  
)
```

3.2 Page settings

```
#set page(  
  // Header and footer  
  header: [  
    #image("assets/logo.svg")  
  ],  
  
  footer: [  
    Typst presentation  
  ]  
)
```

4. Scripting

4. Scripting

- All Typst contents are functions: `#text[]`, `#link[]`, `#image[]`, ...
- The contents can be stored in variables

4.1 Variables & Conditional statements

<pre>#let x = 0 #if x == 1 { text("x is 1") } else if x == 0 { text("x is 0") } else { text("x is something else") }</pre>	x is 0
--	---------------

4.2 Loops

<pre>#for i in range(1, 11) { text[#i] }</pre>	12345678910
--	-------------

4.2 Loops

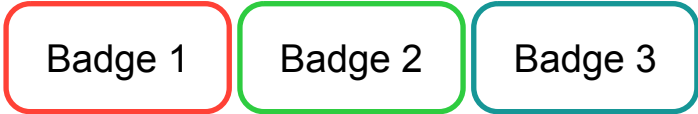
```
#let c = 7
#for i in range(1, 11) {
  [#i $times$ #c = #(i * c) \ ]
}
```

```
1 × 7 = 7
2 × 7 = 14
3 × 7 = 21
4 × 7 = 28
5 × 7 = 35
6 × 7 = 42
7 × 7 = 49
8 × 7 = 56
9 × 7 = 63
10 × 7 = 70
```

4.3 Functions

```
#let badge = (content, color: red) => {  
  box(  
    stroke: 2pt + color,  
    inset: 15pt,  
    radius: 10pt,  
  )[#content]  
}
```

```
#badge[Badge 1]  
#badge(color: green)[Badge 2]  
#badge(color: rgb("#169696"))[Badge 3]
```



Badge 1

Badge 2

Badge 3

4.3 Functions

```
#let isPrime = x => {  
  let half = int(x / 2);  
  for j in range(2, half) {  
    if (calc.rem(x, j) == 0) {  
      return false;  
    }  
  }  
  return true;  
}  
  
#for i in range(2, 500) {  
  if isPrime(i) {  
    text[#i, ]  
  }  
}
```

4.3 Functions

```
#let isPrime = x => {  
  let half = int(x / 2);  
  for j in range(2, half) {  
    if (calc.rem(x, j) == 0) {  
      return false;  
    }  
  }  
  return true;  
}  
  
#for i in range(2, 500) {  
  if isPrime(i) {  
    text[#i, ]  
  }  
}
```

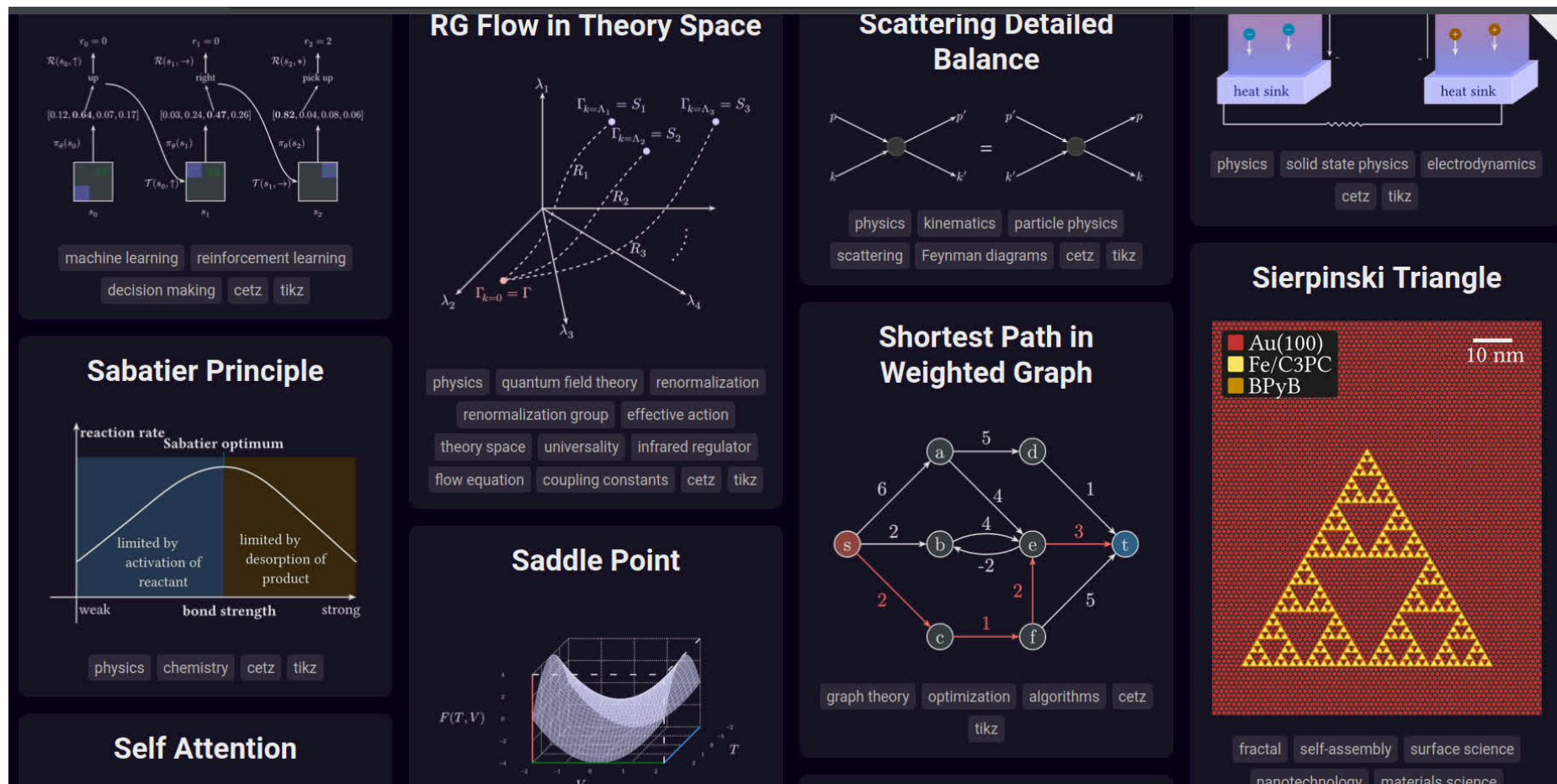
2, 3, 4, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53,
59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109,
113, 127, 131, 137, 139, 149, 151, 157, 163, 167, 173,
179, 181, 191, 193, 197, 199, 211, 223, 227, 229, 233,
239, 241, 251, 257, 263, 269, 271, 277, 281, 283, 293,
307, 311, 313, 317, 331, 337, 347, 349, 353, 359, 367,
373, 379, 383, 389, 397, 401, 409, 419, 421, 431, 433,
439, 443, 449, 457, 461, 463, 467, 479, 487, 491, 499,

5. Packages

5. Packages

- The functionality of Typst can be extended via packages: <https://typst.app/universe>
- For export formats, templates, figures, advanced graphics, data visualization, etc.

5.1 CeTz: For drawing

<https://diagrams.janosh.dev/>


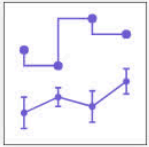
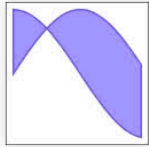
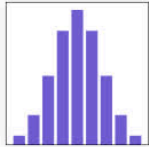
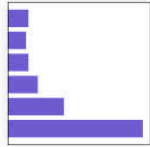
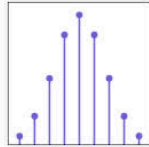
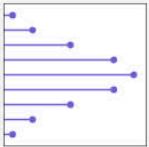
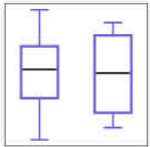
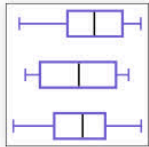
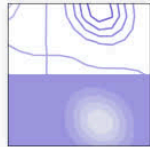
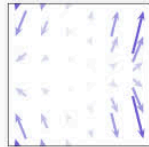
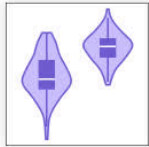
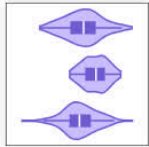
5.2 Lilaq: For data visualization

<https://lilaq.org>

 **Lilaq** Docs News Themes

Ask  

Plot types

 plot	 scatter	 fill-between	 bar	 hbar	 stem
 hstem	 boxplot	 hboxplot	 colormesh	 contour	 quiver
		 violin	 hviolin		

5.3 Touying: For presentation

<https://typst.app/universe/package/touying>

The screenshot displays the 'Touying' package page on the Typst.app website. The page features a navigation bar with links to 'Docs', 'News', 'Themes', 'Universe', and 'Reference'. The main content area shows four preview cards for different presentation themes:

- Dewdrop**: Features elegant navigation with two modes: sidebar and mini-slides. Inspired by the BeamerTheme project. Includes a 'View Docs +' button.
- University**: An aesthetically pleasing theme courtesy of Pol Dellaiera. Well-suited for academic or institutional presentations. Includes a 'View Docs +' button.
- Aqua**: Created by @pride7, featuring beautiful backgrounds made with Typst's built-in visualization capabilities.
- Stargazer**: Originally created by Coekjan for the touying-buaa project. Versatile and visually striking.

Each preview card shows a sample slide layout with fields for Title, Subtitle, Authors, Institution, and Date.

5.4 Alchemist: For creating chemical diagrams

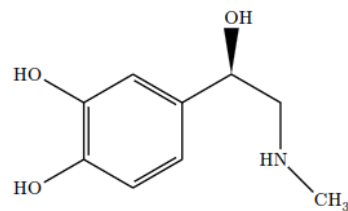
<https://typst.app/universe/package/alchemist>

II Drawing molecules

II.10 Examples

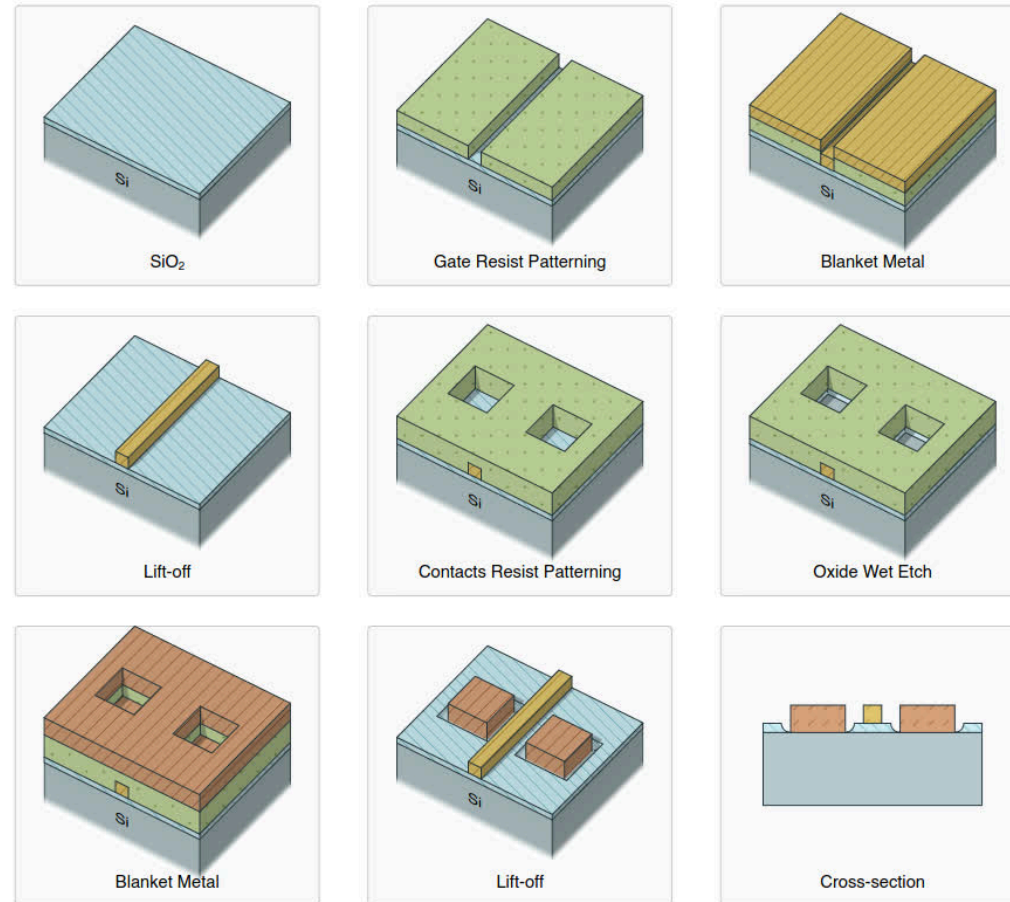
II.10.6 Adrenaline

```
#skeletalize({  
  cycle(6, {  
    branch({  
      single()  
      fragment("HO")  
    })  
    single()  
    double()  
    cycle(6, {  
      single(stroke:transparent)  
      single(  
        stroke:transparent,  
        to: 1  
      )  
      fragment("HN")  
      branch({  
        single(angle:-1)  
        fragment("CH_3")  
      })  
      single(from:1)  
      single()  
      branch({  
        cram-filled-left(angle: 2)  
        fragment("OH")  
      })  
      single()  
    })  
    single()  
    double()  
    single()  
    branch({  
      single()  
      fragment("HO")  
    })  
    double()  
  })  
})
```



5.5 Mesa: Draw semiconductor devices and fabrication process

<https://typst.app/universe/package/mesa>

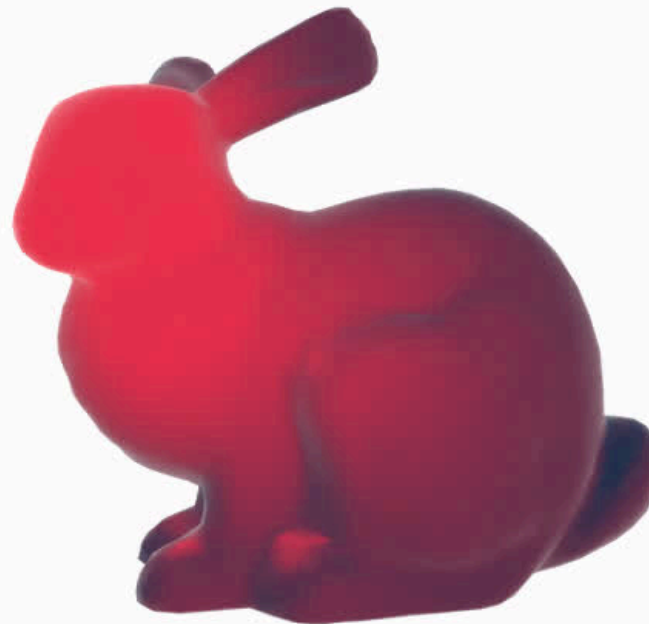


5.6 Maquette: Render 3D models as SVG or PNG images

<https://typst.app/universe/package/maquette>

```
#import "@preview/maquette:0.1.3": render-obj
#let bunny = read("examples/data/bunny.obj",
encoding: none)

#render-obj(bunny,
  up: (0, 1, 0),
  azimuth: 180,
  distance: 0.25,
  background: none,
  lights: (
    (type: "positional",
      vector: (-0.1, 0.14, -0.04),
      color: "#ff0000", intensity: 3.0),
  ),
  sss: (intensity: 4, power: 3.5, distortion:
0.2),
  antialias: 4)
```





Thank you

